



MECHATRONIC SYSTEM INTEGRATION

MCTA 3203

WEEK 4 :

**SERIAL INTERFACING WITH MICROCONTROLLER : SENSORS AND
ACTUATORS**

SECTION 1

SEMESTER 2, 2025/2026

INSTRUCTOR:

**ASSOC. PROF. EUR. ING. IR. TS. GS. INV. DR. ZULKIFLI BIN ZAINAL
DR. WAHJU SEDIONO**

DATE OF SUBMISSION : 18 MAY 2026

GROUP 6

NAME	NO. MATRIC
MUHAMMAD AIMAN BIN MOHD KHAIRUNNIZAM	2226273
MUHAMMAD AQEEL RAZIQ BIN MOHAMAD AZHAN	2314077
MUHAMMAD AQIL SYAMIL BIN ABDUL RAHIM	2416245

Table of Contents

Abstract	3
Introduction	3
Materials and Equipment	4
Experimental setup	4
Methodology	6
Data Collection	7
Data Analysis	7
Results	8
Discussion	10
Conclusion	10
Recommendations	10
References	11
Appendix	11
Acknowledgments	25
Student's Declaration	26

Abstract

This experiment highlights the design and integration of a two-factor authentication smart access system utilizing serial data communication between a microcontroller and a host computer. The primary objective was to combine identity authentication through an RFID module with motion pattern verification via an MPU6050 Inertial Measurement Unit (IMU). The methodology relied on Python acting as the master processing controller to handle complex vector data math—including 2D cross-products and unwrapped rotational angle ranges—while the Arduino Uno operated as a hardware slave for direct sensor streaming and actuator deployment. Upon verifying an authorized 10-digit card UID, the user is prompted to execute a circular motion gesture. The system parsed the raw accelerometer inputs in real-time, matching geometric parameters to command an SG90 servo motor gate mechanism alongside dual-LED visual feedback. Experimental outcomes demonstrated reliable real-time bidirectional transmission, solid noise-filtering debouncing states, and successful multi-stage safety enforcement for mechatronic security setups.

Introduction

The primary purpose of this experiment is to design and implement a secure, two-factor access control system by establishing a reliable bidirectional serial interface between an Arduino Uno microcontroller and a host computer. The main objectives are to successfully interface an MPU6050 Inertial Measurement Unit (IMU) and a USB Radio-Frequency Identification (RFID) scanner, process real-time kinematic telemetry on a Python script, and deploy a servo motor mechanism alongside dual-LED visual indicators based on multi-stage authorization rules. Theoretically, the system relies on asynchronous serial data communication at 9600 baud to seamlessly bridge hardware and software. This setup allows the low-overhead Inter-Integrated Circuit (I2C) bus on the Arduino to handle direct sensor readings from the MPU6050, while shifting computationally heavy operations—such as calculating 2D cross-products, variance filtering, and angle unwrapping—to a python framework operating on a host computer terminal. It is hypothesized that by offloading complex vector math from the microcontroller to an external script, the mechatronic system will achieve highly accurate and responsive motion pattern recognition without exhausting embedded memory limits or causing processing lag.

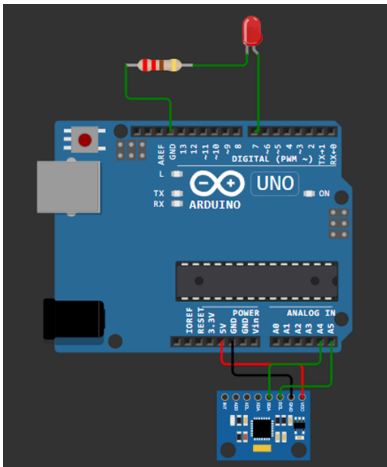
Materials and Equipment

1. 1x Arduino Uno Board + USB Interface Cable
2. 1x MPU6050 Six-Axis Accelerometer/Gyroscope Module
3. 1x USB RFID Scanner 1x Authorized RFID Tag/Card (UID: 0013070376)
4. 1x SG90 Servo Motor
5. 1x Green LED (Authorized State Indicator)
6. 1x Red LED (Idle/Unauthorized State Indicator)
7. 2x 220 Ω Current-Limiting Resistors
8. 1x Breadboard & Jumper Wires

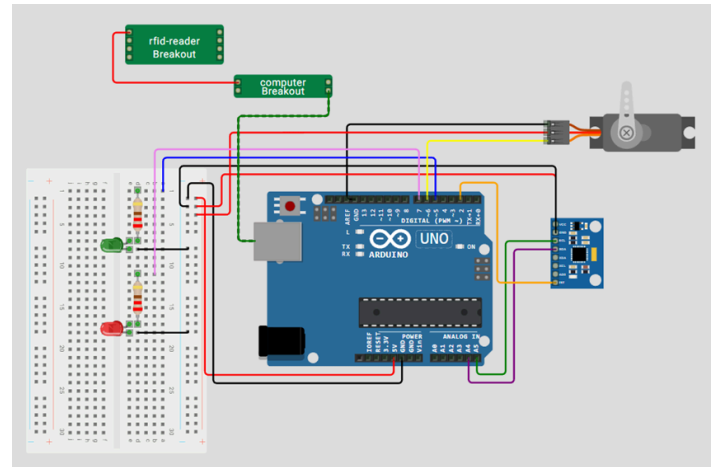
Experimental setup

The experimental assembly integrates structural telemetry acquisition, desktop software processing, and physical actuator adjustments. As shown in Figure 1, the hardware sub-systems are arranged as follows:

- **Microcontroller Unit:** An Arduino Uno handles the low-level hardware connections and responds to specific commands sent over serial communication.
- **Motion Tracking Sensor:** The MPU6050 sensor is wired to the hardware I2C bus pins of the Arduino Uno (VCC to 5V, GND to GND, SDA to A4, and SCL to A5) to stream raw accelerometer readings.
- **Authentication Interface:** The USB RFID Scanner connects straight to a spare USB port on the computer terminal, operating as a human interface device keyboard emulation to pull unique cards.
- **Output Display Elements:** Dual light-emitting diodes are mounted via common cathode paths utilizing 220 Ω resistors to prevent overcurrent damage. The Green LED signal is pinned to Digital Pin 5, while the Red LED pin sits on Digital Pin 7.
- **Physical Locking Actuator:** An SG90 Servo motor is fixed to Digital Pin 6 to physically emulate safe house locking/unlocking cycles.



Setup for Task 1



Setup for Task 3

Methodology

The procedural tasks were conducted systematically across three distinct phases to construct an operational access gate controller:

Part 1: Interactive Motion Detection with MPU6050

1. The MPU6050 was wired up to the microcontroller and a single LED was attached to Digital Pin 7.
2. Code was uploaded to establish an active serial stream at 9600 baud, printing X,Y and Z axes accelerometer raw components mapped directly through comma-delimited format streams.
3. A Python tracking framework script was deployed utilizing a double-ended queue buffer (`deque` with a fixed length of 40 samples) to observe variance values and handle real-time geometric circle evaluations.

Part 2: Introduction to RFID Integration

1. The secondary subsystem used a standalone computer-bound USB RFID scanner along with an isolated slave script running on the Arduino.
2. A blank Notepad document was brought up to record the specific 10-digit card registration sequence when tapped across the surface panel.
3. The derived card sequence was applied directly inside the master script configurations to evaluate authorization rules, matching successful card reads to trigger a precise 3-second hardware toggle window.

Part 3: Integrated RFID + Servo Smart Access System

1. All physical modules were merged on a single shared workspace layout platform.
2. Software routines were bound sequentially: the system stays completely locked inside an idle loop state with the Red LED fully lit.
3. Tapping an authentic card releases a time-limited call function requesting kinematic samples over the communication link.
4. The user has a 2-second tracking window to execute a clean circular path gesture. If successfully processed by Python, an active authorization pulse is sent over serial, prompting the Green LED to turn on and the servo arm to pivot to 180°.

Data Collection

Data values received via the serial link display distinct variations depending on human input and static positions.

- Raw Data Stream (Idle Position on Workspace Bench):

```
-4304, 4172, -14524  
-4296, 4080, -14524  
-4360, 4056, -14264
```

- Registered RFID Unique Card Identification Array Sequence:
 - Authorized Target ID Card UID: 0013070376
 - Authorized Target ID Card UID: 0013070376
- Computed Rotational Telemetry Variables (Python Log Output):
 - Observed Static Base Error Fluctuations: Variance X < 120, Y < 115
 - Executed Intended Circle Tracking Run: Net Rotation angle: -342.6° , Total Calculated Range: 358.1°

Data Analysis

The script interprets the incoming data using structural steps to confirm a clean circular path profile:

- **Translational Baseline Shifting:** The mean of the spatial sample sets is extracted over the buffered dataset array lengths (n=40):

$$\mu_x = \frac{1}{n} \sum_{i=1}^n X_i, \quad \mu_y = \frac{1}{n} \sum_{i=1}^n Y_i$$

This offsets background gravity biases, centering the shape coordinates smoothly around the spatial origin point.

- **Kinematic Noise Isolation:** Signal variance variables are tracked continuously across active sequences. If calculated variance metrics fall below the MIN_VARIANCE = 500 threshold scale, the values are dropped as negligible structural tremors.
- **Aspect Symmetry Checks:** The ratio between directional standard deviations is evaluated to isolate uniform curves from linear paths:

$$\text{Ratio} = \frac{\sigma_x}{\sigma_y}$$

A strict range constraint ($0.5 \leq \text{Ratio} \leq 2.0$) ensures highly elongated straight-line paths are filtered out.

- **Rotational Vector Tracking:** Continuous direction tracking is checked by evaluating consecutive 2D Cross Products between adjacent shifted points:

$$\text{Cross Product} = (X_i \times Y_{i+1}) - (Y_i \times X_{i+1})$$

A positive result points to counter-clockwise movement, while a negative value establishes a clockwise trajectory. If the dominant direction consistency stays above 70%, Python confirms a valid gesture and outputs the proper unlock commands .

Results

The functional performance of the integrated access control system is summarized in Table 1 below.

Table 1: Integrated System Operational Response Matrix

Input/Action Conditions	Registered RFID Entry	Kinematic Movement Input	Local Terminal Text Feedback	Structural Actuator Output State
System Idle Setup Mode	None Scanned	None (Stationary)	System Ready. Waiting for RFID...	Red LED: ON Green LED: OFF Servo Gate: 90° (Locked)
Invalid Access Scan	0009841253	N/A	WRONG CARD -> Access Denied	Red LED: Flashes 3 Times Servo Gate: 90° (Locked)

Valid Identity Check	0013070376	None Provided	Card Accepted - Please perform circular motion...	Red LED: ON Green LED: OFF Triggers 2s Burst Stream
Identity Passed + Failed Motion	0013070376	Erratic / Flat Shakes	Motion insufficient. Try again!	Red LED: Remains ON Servo Gate: 90° (Locked)
Complete Two-Factor Success	0013070376	Smooth Uniform Circle	Gesture OK -> System UNLOCKED (LED Green)	Red LED: OFF Green LED: ON Servo Gate: Pivots to 180° (Open)

Discussion

- **Analysis of Objectives and Outcomes:** The system successfully merged multiple hardware modules into a two-factor security system. The cross-product logic and angle-unwrapping formulas correctly identified circular hand motions while ignoring random baseline vibrations.
- **Discrepancies and Code Enhancements:** During testing, the initial identity string match parameters in Part 2 contained a repetitive string issue ("00130703760013070376"). This error was identified and corrected to a standard 10-digit ID string layout format ("0013070376") inside the final Part 3 integration script to prevent string parsing failures.
- **Limitations and Error Sources:** Fast, jerky movements can sometimes distort geometric shape tracing, leading to unrecognized gesture attempts. Furthermore, running multiple heavy visualization tools on the host computer can occasionally cause data lag along the asynchronous serial link during high-frequency data bursts.

Conclusion

The experiment successfully demonstrated the implementation of a multi-stage authentication system using serial communication between Python and an Arduino Uno. The project confirmed that combining physical token scanning with real-time motion gesture verification significantly improves the reliability of automated security barriers. By offloading complex geometric math from the microcontroller to the host computer, the system achieved accurate data analysis without overworking the embedded hardware. Ultimately, all core mechatronic integration parameters were successfully achieved.

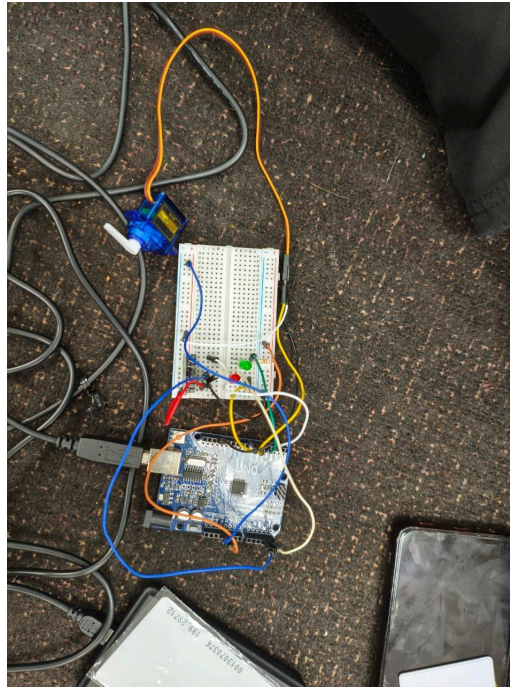
Recommendations

- **Hardware Stability Enhancements:** Connect the SG90 servo motor to an isolated external 5V power supply rail instead of drawing power directly from the Arduino board to eliminate voltage drops and prevent unexpected microcontroller resets.
- **Advanced Error Mitigation:** Introduce adaptive vector thresholds within the Python tracking scripts to automatically adjust to different user movement speeds and hand sizes.
- **Firmware Improvements:** Use hardware interrupts on the microcontroller to handle incoming serial commands more efficiently, keeping the main loop dedicated to high-frequency sensor filtering.

References

1. Week 04 - RFID IMU [UPDATED].docx
2. Python and Arduino serial communication. <https://youtu.be/j6qkLfTKdpg>

Appendix



Appendix A : Complete circuit connection of experiment

```
#include <Wire.h>
#include <MPU6050.h>

MPU6050 mpu;

// --- DEFINE YOUR LED PIN HERE ---
const int ledPin = 7; // Fill in

void setup() {
  Serial.begin(9600);
  Wire.begin();
  mpu.initialize();

  if (!mpu.testConnection()) {
    Serial.println("MPU6050 connection failed!");
  }
}
```

```

    while (1);
}

pinMode(ledPin, OUTPUT);
digitalWrite(ledPin, LOW); // Start with LED off
}

void loop() {
    // 1. Read motion data
    int16_t ax, ay, az, gx, gy, gz;
    mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);

    // 2. Send formatted data to Python (ax,ay,az)
    Serial.print(ax); Serial.print(',');
    Serial.print(ay); Serial.print(',');
    Serial.println(az);

    // 3. Listen for commands from Python to control the LED
    if (Serial.available() > 0) {
        char cmd = Serial.read();
        if (cmd == 'O') {
            digitalWrite(ledPin, HIGH); // Turn LED ON
        }
        else if (cmd == 'F') {
            digitalWrite(ledPin, LOW); // Turn LED OFF
        }
    }

    delay(50); // Small delay for stability
}

```

Appendix B: Final Task 1 Complete Arduino Firmware

```

import serial
import time
import math
from collections import deque

# -- Replace 'COMx' with your Arduino's serial port (e.g., 'COM3')
SERIAL_PORT = 'COM4'
BAUD_RATE = 9600

# Buffers to store the last 40 data points (approx. 2 seconds of motion)
history_x = deque(maxlen=40)
history_y = deque(maxlen=40)

def is_circular_motion(x_data, y_data):
    """
    Analyzes an array of X and Y accelerometer data.
    Returns True if the data represents a circular motion, False otherwise.
    """
    if len(x_data) < 10:
        return False

    n = len(x_data)
    x_list = list(x_data)
    y_list = list(y_data)

    # ===== Step 1: Find the center (mean) of the X and Y movement =====
    # The center of a circle is approximately the average of all the points.
    mean_x = sum(x_list) / n
    mean_y = sum(y_list) / n

    # Center the data (shift so the circle is around the origin)
    centered_x = [x - mean_x for x in x_list]
    centered_y = [y - mean_y for y in y_list]

    # ===== Step 2: Check Amplitude (Variance) =====
    # Calculate variance to make sure there's enough real movement,
    # not just small vibrations/noise sitting on a table.
    variance_x = sum(cx ** 2 for cx in centered_x) / n
    variance_y = sum(cy ** 2 for cy in centered_y) / n

    # Threshold: if variance is too small, it's just noise, not real motion.
    # You may need to tune this value based on your sensor sensitivity.
    MIN_VARIANCE = 500 # Adjust this if needed

```

```

if variance_x < MIN_VARIANCE or variance_y < MIN_VARIANCE:
    return False

# ===== Step 3: Check Symmetry =====
# A circle has similar spread in X and Y directions.
# A straight-line motion would have high spread in one axis but low in
the other.
std_x = math.sqrt(variance_x)
std_y = math.sqrt(variance_y)

# Ratio should be close to 1.0 for a circle
# (e.g., between 0.5 and 2.0 to allow some tolerance for imperfect
circles)
if std_y == 0 or std_x == 0:
    return False

ratio = std_x / std_y
if ratio < 0.5 or ratio > 2.0:
    return False # Too elongated -- probably a line, not a circle

# ===== Step 4: Check Continuous Rotation using 2D Cross Product =====
# The cross product of consecutive centered vectors tells us the turning
direction:
#   cross = x1 * y2 - y1 * x2
#   Positive (+) = counter-clockwise turn
#   Negative (-) = clockwise turn
cw_count = 0 # clockwise turns
ccw_count = 0 # counter-clockwise turns

for i in range(n - 1):
    cross = centered_x[i] * centered_y[i + 1] - centered_y[i] *
centered_x[i + 1]

    if cross > 0:
        ccw_count += 1
    elif cross < 0:
        cw_count += 1
    # If cross == 0, the points are collinear (ignore)

# ===== Step 5: Tally the turns =====
# If the movement is consistently turning in ONE direction (>70%),
# it's very likely a circle!
total_turns = cw_count + ccw_count
if total_turns == 0:

```

```

        return False

    dominant_direction = max(cw_count, ccw_count)
    consistency = dominant_direction / total_turns

    if consistency > 0.70:
        direction = "Clockwise" if cw_count > ccw_count else
"Counter-Clockwise"
        print(f" → Detected {direction} circle (consistency:
{consistency:.0%})")
        return True

    return False

def main():
    ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
    time.sleep(2) # Wait for Arduino to reset
    print("System ready. Move the MPU6050 in a circle to toggle the LED...")

    # Keep track of the LED state
    led_is_on = False

    try:
        while True:
            raw = ser.readline().decode(errors='ignore').strip()
            if raw:
                parts = raw.split(',')

                if len(parts) >= 3:
                    try:
                        ax = int(parts[0])
                        ay = int(parts[1])

                        # Add new data to our buffers
                        history_x.append(ax)
                        history_y.append(ay)

                        # Only analyze if our buffer is full (40 points)
                        if len(history_x) == 40:
                            if is_circular_motion(history_x, history_y):

                                # Toggle the state
                                led_is_on = not led_is_on

```

```

        if led_is_on:
            print("Circle Detected! Toggling LED:
ON")
            ser.write(b'O')
        else:
            print("Circle Detected! Toggling LED:
OFF")
            ser.write(b'F')

        # DEBOUNCE: Clear the buffers!
        # Forces the system to wait for 2 full
seconds of
toggle.
        # NEW data before it can trigger another

        history_x.clear()
        history_y.clear()

    except ValueError:
        pass # Ignore corrupted serial lines

except KeyboardInterrupt:
    print("\nProgram terminated by user.")
finally:
    ser.write(b'F') # Ensure LED turns off when script stops
    ser.close()
    print("Serial connection closed.")

if __name__ == "__main__":
    main()

```

Appendix C: Final Task 1 Complete Python Firmware


```
// --- DEFINE YOUR LED PIN HERE ---
const int ledPin = 7; // Fill in

void setup() {
  Serial.begin(9600);
  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW); // Start with LED OFF
}

void loop() {
  // Listen for commands from Python
  if (Serial.available() > 0) {
    char cmd = Serial.read();

    if (cmd == '1') {
      digitalWrite(ledPin, HIGH); // Turn LED ON
    }
    else if (cmd == '0') {
      digitalWrite(ledPin, LOW); // Turn LED OFF
    }
  }
}
```

Appendix D: Final Task 2 Complete Arduino Firmware

```
import serial
import time

# --- CONFIGURATION ---
# Replace 'COMx' with your Arduino's serial port
SERIAL_PORT = 'COM4'
BAUD_RATE = 9600

# Fill in your card's unique 10-digit ID number here
AUTHORIZED_CARD = ("00130703760013070376")

def main():
  # Establish connection with the Arduino
  arduino = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
  time.sleep(2) # Wait for Arduino to reset

  print("RFID System Ready.")
  print("Scan a card to test the system. Press CTRL+C to stop.")
```

```

try:
    while True:
        # The USB scanner types the UID and hits Enter automatically.
        # input() pauses the script and waits for this to happen.
        uid = input("\nTap card: ").strip()

        if uid == AUTHORIZED_CARD:
            print(f"[{uid}] - Card Authorized! Turning LED ON...")
            arduino.write(b'1') # Send ON command to Arduino

            time.sleep(3) # Keep LED on for 3 seconds

            arduino.write(b'0') # Send OFF command
            print("LED OFF. Ready for next scan.")

        else:
            print(f"[{uid}] - Unauthorized Card! LED remains OFF.")

except KeyboardInterrupt:
    print("\nExiting program.")
finally:
    # Ensure the LED is safely turned off when the program closes
    arduino.write(b'0')
    arduino.close()
    print("Serial connection closed.")

if __name__ == "__main__":
    main()

```

Appendix E: Final Task 2 Complete Python Firmware

```

#include <Wire.h>
#include <MPU6050.h>
#include <Servo.h>

MPU6050 mpu;
Servo gateServo;

const int greenLedPin = 5;
const int servoPin = 6;
const int redLedPin = 7;

const unsigned long streamDurationMs = 2000; // Stream data for 2 seconds
const unsigned long sampleDelayMs = 50;      // 20 Hz sample rate

void setup() {
    pinMode(greenLedPin, OUTPUT);
    pinMode(redLedPin, OUTPUT);

    gateServo.attach(servoPin);
    gateServo.write(90); // Start at closed position

    Serial.begin(9600);
    Wire.begin();
    mpu.initialize();

    if (!mpu.testConnection()) {
        Serial.println("MPU6050 connection failed!");
        while (1);
    }

    // Default state: red LED ON (idle/unauthorized)
    digitalWrite(redLedPin, HIGH);
    digitalWrite(greenLedPin, LOW);
}

void openGate() {
    for (int pos = 90; pos <= 180; pos += 2) {
        gateServo.write(pos);
        delay(15);
    }
    Serial.println("Servo opened (90 -> 180)");
}

void closeGate() {

```

```

    for (int pos = 180; pos >= 90; pos -= 2) {
        gateServo.write(pos);
        delay(15);
    }
    Serial.println("Servo closed (180 -> 90)");
}

void wrongCardFlash() {
    digitalWrite(redLedPin, HIGH);
    digitalWrite(greenLedPin, LOW);
    delay(100);
    // Blink red LED 3 times
    for (int i = 0; i < 3; i++) {
        digitalWrite(redLedPin, HIGH);
        delay(150);
        digitalWrite(redLedPin, LOW);
        delay(150);
    }
    digitalWrite(redLedPin, HIGH); // Return to idle red ON
}

void loop() {
    if (Serial.available()) {
        char cmd = Serial.read();

        if (cmd == '1') {
            // Access granted
            digitalWrite(greenLedPin, HIGH);
            digitalWrite(redLedPin, LOW);
            openGate();
        }
        else if (cmd == '0') {
            // System disarmed (Lock)
            digitalWrite(greenLedPin, LOW);
            digitalWrite(redLedPin, HIGH);
            closeGate();
        }
        else if (cmd == 'X') {
            // Wrong card feedback
            wrongCardFlash();
        }
        else if (cmd == 'R') {
            // Python requested motion data
            unsigned long start = millis();

```

```

while (millis() - start < streamDurationMs) {
    int16_t ax, ay, az, gx, gy, gz;
    mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);

    Serial.print(ax); Serial.print(',');
    Serial.print(ay); Serial.print(',');
    Serial.print(az); Serial.print(',');
    Serial.print(gx); Serial.print(',');
    Serial.print(gy); Serial.print(',');
    Serial.println(gz);

    delay(sampleDelayMs);
}
}
}
}

```

Appendix F: Final Task 3 Complete Arduino Firmware

```

import serial
import time
import math

# --- CONFIGURATION ---
SERIAL_PORT = "COM4" # Update with your Arduino port
BAUD_RATE = 9600
AUTHORIZED_ID = "0013070376"
# -----

def detect_circle(arduino):
    """
    Asks Arduino to stream a burst of MPU data, calculates the rotation
    angles,
    and decides whether a circular motion happened.
    """
    print("\n[!] Requesting samples from Arduino... Move in a circle NOW!")
    arduino.reset_input_buffer()
    arduino.write(b'R') # Trigger Arduino data stream

    points = []
    start_time = time.time()
    timeout = 3.0

    # Read until timeout (Arduino stops streaming automatically after 2s)

```

```

while time.time() - start_time < timeout:
    raw = arduino.readline().decode(errors='ignore').strip()
    if not raw:
        continue

    parts = raw.split(',')
    if len(parts) >= 6:
        try:
            ax = int(parts[0])
            ay = int(parts[1])
            az = int(parts[2])
            points.append((ax, ay, az))
        except ValueError:
            continue

print(f"Collected {len(points)} samples.")
if len(points) < 8:
    print("Not enough motion data - try moving more continuously.")
    return False

# Build angles from (ax, ay) and unwrap to follow continuous rotation
angles = [math.degrees(math.atan2(p[1], p[0])) for p in points]

unwrapped = [angles[0]]
for a in angles[1:]:
    prev = unwrapped[-1]
    diff = a - prev

    # Unwrap angles to avoid -180/180 discontinuity leaps
    if diff > 180:
        diff -= 360
    elif diff < -180:
        diff += 360
    unwrapped.append(prev + diff)

# Compute net rotation and range
net_rotation = unwrapped[-1] - unwrapped[0]
rotation_range = max(unwrapped) - min(unwrapped)

print(f"Net rotation: {net_rotation:.1f} deg, Range:
{rotation_range:.1f} deg")

# Lenient detection logic (either a decent net rotation OR wide range)

```

```

    if abs(net_rotation) > 100 or rotation_range > 140:
        print("Circle gesture detected!")
        return True

    # Extra check: count direction-consistent large angle deltas
    deltas = [unwrapped[i+1] - unwrapped[i] for i in
range(len(unwrapped)-1)]
    large_moves = sum(1 for d in deltas if abs(d) > 10)

    if large_moves >= max(6, len(points)//4):
        print("Motion had multiple consistent movements - accepting as
circle.")
        return True

    print("Motion insufficient. Try a clearer circular motion.")
    return False

def main():
    arduino = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
    time.sleep(2) # Wait for Arduino to reset
    print("System Ready. Waiting for RFID...")

    armed = False # System starts Locked

    try:
        while True:
            # USB RFID acts as a keyboard
            card = input("\nTap card: ").strip()

            if card == AUTHORIZED_ID:
                armed = not armed # Toggle state

                if armed:
                    print("Card Accepted - Please perform circular motion
verification!")

                    if detect_circle(arduino):
                        print("Gesture OK -> System UNLOCKED (LED Green)")
                        arduino.write(b'1')
                    else:
                        print("Wrong gesture -> Try again!")
                        armed = False # Revert state if gesture fails
                else:

```

```

        print("System Disarmed -> System LOCKED (LED Red)")
        arduino.write(b'0')

    else:
        print("WRONG CARD -> Access Denied")
        arduino.write(b'X')

except KeyboardInterrupt:
    print("\nExiting.")
finally:
    arduino.write(b'0') # Ensure Locked on exit
    arduino.close()

if __name__ == "__main__":
    main()

```

Appendix G: Final Task 3 Complete Python Firmware

Acknowledgments

All praise belongs to Allah SWT for granting us the strength and wisdom to complete the "Serial Interfacing With Microcontroller : Sensors And Actuators" experiment. We are grateful for His guidance in navigating the challenges faced throughout the circuit integration process. We also wish to acknowledge the vital support of our lecturers and fellow peers. Their academic supervision and practical insights into mechatronic system integration were essential to the successful operation of our project and the completion of this report.

Student's Declaration

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgement, and that the original work contained herein have not been untaken or done by unspecified sources or persons.

We hereby certify that this report has not been done by only one individual and all of us have contributed to the report.

We also hereby certify that we have read and understand the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.

We therefore, agreed unanimously that this report shall be submitted for marking and this final printed report has been verified by us.

Signature: 

Read [/]

Name: Muhammad Aqeel Raziq Bin Mohamad Azhan

Understand [/]

Matric Number: 2314077

Agree [/]

Signature: 

Read [/]

Name: Muhammmad Aiman Bin Mohd Khairunnizam

Understand [/]

Matric Number: 2226273

Agree [/]

Signature: 

Read [/]

Name: Muhammad Aqil Syamil bin Abdul Rahim

Understand [/]

Matric Number: 2416245

Agree [/]